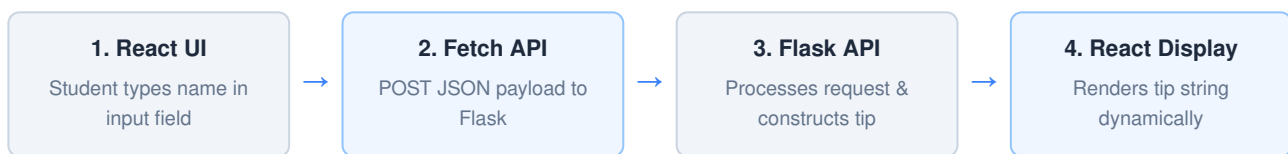


Building a Full-Stack Mini App

Learn how React talks to Flask one step at a time through a Study Tip Generator

1. What Are We Building & Architecture Overview

We are constructing a client-server mini application where a student enters their name on a React frontend interface, sends an asynchronous HTTP POST request, and receives a personalized study tip generated by a Flask backend.



2. Project Structure & Prerequisites

Ensure you have **Python 3**, **Node.js**, and a code editor (e.g., VS Code) installed. Organize your directory into clear frontend and backend folders:

```
my-app/  
├── backend/  
│   ├── app.py  
│   └── requirements.txt  
├── frontend/  
├── package.json  
├── src/  
└── App.js
```

```
# Environment Setup Commands  
$ cd backend  
$ pip install flask flask-cors  
  
$ cd ../frontend  
$ npx create-react-app .  
$ npm start
```

3. Step 1: Building the Flask Backend

Flask is a lightweight Python web framework used here to expose an API endpoint (/api/tip).

```
backend/app.py  
  
from flask import Flask, request, jsonify  
from flask_cors import CORS  
  
app = Flask(__name__)  
CORS(app) # Enables Cross-Origin Resource Sharing
```

```
@app.route('/api/tip', methods=['POST'])
def get_tip():
    data = request.get_json()
    name = data.get('name', 'Student')
    tip = f"Hey {name}, try the Pomodoro method today!"
    return jsonify({'tip': tip})

if __name__ == '__main__':
    app.run(debug=True, port=5000)
```

- **Route & Method:** `@app.route('/api/tip', methods=['POST'])` defines the HTTP endpoint path and restricts it to POST actions.
- **JSON Processing:** `request.get_json()` parses payload data; `jsonify()` serializes Python dictionaries into standard JSON API responses.
- **CORS Support:** `flask-cors` permits secure browser requests from React (port 3000) to Flask (port 5000).

4. Testing API & Step 2: Building React Frontend

Test endpoint using cURL before frontend integration:

```
$ curl -X POST http://127.0.0.1:5000/api/tip -H "Content-Type: application/json" -d '{"name": "Asha"}'
{"tip": "Hey Asha, try the Pomodoro method today!"}
```

frontend/src/App.js

```
import React, { useState } from 'react';

function App() {
    const [name, setName] = useState('');
    const [tip, setTip] = useState('');

    const handleSubmit = async (e) => {
        e.preventDefault();
        const response = await fetch('http://127.0.0.1:5000/api/tip', {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({ name })
        });
        const data = await response.json();
        setTip(data.tip);
    };

    return (
        <div style={{ padding: '20px' }}>
            <h2>Study Tip Generator</h2>
            <form onSubmit={handleSubmit}>
                <input type="text" value={name} onChange={(e) => setName(e.target.value)}
placeholder="Enter your name..." />
                <button type="submit">Get My Tip</button>
            </form>
            {tip && <p style={{ marginTop: '15px' }}>{tip}</p>}
        </div>
    );
}
```

```
);  
}
```

5. Troubleshooting & Debugging Checklist

- **Flask Server Not Running:** Ensure `python app.py` is actively listening in terminal before submitting from React.
- **CORS Blocked Error:** Verify `CORS(app)` is initialized in Flask to permit requests from origin `http://localhost:3000`.
- **Network 404 / Port Mismatch:** Confirm standard fetch URL specifies port `5000` and route path `/api/tip`.

STUDENT PRACTICE WORKSHEET

Student Name: _____

Date: _____

Topic: Full-Stack React & Flask Integration | Assessment

Question 1: Conceptual Architecture

Explain the role of the frontend and backend in a full-stack web application. What specific tasks does React perform versus Flask in the Study Tip Generator project?

Question 2: HTTP Protocols & Methods

Why was an HTTP POST request used for fetching the study tip instead of an HTTP GET request? What key information is included in the POST request body?

Question 3: Cross-Origin Resource Sharing (CORS)

Suppose React runs on `http://localhost:3000` and Flask runs on `http://localhost:5000`. What error will the browser display if `flask-cors` is not enabled, and why does the browser enforce this policy?

Question 4: Code Analysis & Debugging

Examine the snippet below from React. Identify **two errors** that will prevent the study tip from properly displaying:

```
const handleSubmit = async (e) => { const response = fetch('http://127.0.0.1:5000/api/tip', { method: 'GET', body: { name: name } }); setTip(response.tip); };
```

Question 5: Feature Expansion Challenge

Write out the Python code modification for `app.py` if the frontend sent both a student's name and their chosen subject (e.g., "Math" or "History") to generate a subject-specific study tip.